

Design Matrices and Model Prediction

30 May 2026 10:03



08 - Design
Matrices ...

08 — Design Matrices and Model Prediction

Purpose

In the previous notebooks, we saw least squares from two useful angles:

- as a way to choose a line that fits points well
- as a projection idea, where a target vector is approximated using vectors we are allowed to combine

This notebook connects that maths to the shape of a machine learning problem.

The main idea is:

A model can make many predictions at once when we arrange the input data into a matrix.

By the end, we want this expression to feel natural:

$$\hat{y} = X\beta$$

where:

- X is the **design matrix**
- β is the vector of **model parameters**
- $\hat{y}$ is the vector of **predictions**

This is where we move from finding β , to using β .

Why this notebook exists

So far, we have mostly worked with small vectors and matrices as mathematical objects.

Machine learning uses the same objects, but with more practical names:

	Maths object	ML / data interpretation	
Same? {	vector	one list of values	Needs a visual?
	matrix	a table of values	
	row	one observation / example	
	column	one feature	
	matrix multiplication	many predictions at once	
	parameter vector	the numbers the model uses to make predictions	

This notebook is a bridge from:

$$\hat{y} = mx + c$$

to:

$$\hat{y} = X\beta$$

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
```

A tiny dataset

We'll use a small toy dataset so the shapes and calculations stay visible.

Imagine x is one input feature, and y is the observed output we are trying to predict.

```
In [2]: x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2.2, 2.8, 3.6, 4.5, 5.1], dtype=float)

print("x: ", x)
print("y: ", y)
```

```
x: [1. 2. 3. 4. 5.]
y: [2.2 2.8 3.6 4.5 5.1]
```

For now, do not worry about finding the best line.

We'll choose a candidate line manually:

$$\hat{y} = mx + c$$

where:

- m is the slope
- c is the intercept
- $\hat{y}$ means "predicted y "

```
In [3]: m = 0.75
c = 1.3

y_hat_direct = m * x + c
print("y_hat_direct: ", y_hat_direct)
```

```
y_hat_direct: [2.05 2.8 3.55 4.3 5.05]
```

One prediction at a time

For a single observation, the model says:

$$\hat{y}_i = c + mx_i$$

The subscript i means "for row i " or "for observation i ".

For the first input value, $x_1 = 1$, the prediction is:

$$\hat{y}_1 = c + mx_1$$

Why swap
to $c + mx$?
Previous notebooks
preferred $mx + c$

```
In [4]: x_1 = x[0]

y_hat_1 = c + m * x_1

print(f"x_1 = {x_1}")
print(f"prediction = {y_hat_1}")
```

```
x_1 = 1.0
prediction = 2.05
```

For several observations, the same calculation repeats:

$$\hat{y}_1 = c + mx_1$$

$$\hat{y}_2 = c + mx_2$$

$$\hat{y}_3 = c + mx_3$$

$$\hat{y}_4 = c + mx_4$$

$$\hat{y}_5 = c + mx_5$$

This kind of repeated calculation is exactly what matrix multiplication is good at.

Making the repeated calculation into a matrix calculation

We want each row to carry enough information to make one prediction.

The line equation is:

$$\hat{y}_i = c + mx_i$$

We can rewrite this as:

$$\hat{y}_i = 1c + x_im$$

That means each row can be written as:

$$[1 \quad x_i]$$

and the model parameters can be written as:

$$\beta = \begin{bmatrix} c \\ m \end{bmatrix}$$

Then one row times the parameter vector gives one prediction:

$$[1 \quad x_i] \begin{bmatrix} c \\ m \end{bmatrix} = 1c + x_im$$

Again, the 1s
are now first.
Might be fine,
might not.
Needs explaining
at least.

The design matrix

For all our observations, the rows stack into a matrix:

$$X = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \\ 1 & x_4 \\ 1 & x_5 \end{bmatrix}$$

Is 'input values' the right term? Feels more like a 'solution shape'

This is called a **design matrix**.

The word "design" is common in statistics and machine learning. It means the matrix that contains the input values in the shape needed by the model.

```
In [5]: # np.ones_like(x) creates an array of ones with the same shape as x.
# We use this for the intercept column.

intercept_column = np.ones_like(x)

# np.column_stack(...) takes one-dimensional arrays and stacks them as columns.
# Here, the first column is all ones, and the second column is x.

X = np.column_stack([intercept_column, x])

X
```

```
Out[5]: array([[1., 1.],
               [1., 2.],
               [1., 3.],
               [1., 4.],
               [1., 5.]])
```

The first column is all ones. This is not random decoration.

The column of ones lets the intercept c participate in matrix multiplication.

Each row says:

$$\begin{bmatrix} 1 & x_i \end{bmatrix} \begin{bmatrix} c \\ m \end{bmatrix} = 1c + x_i m$$

Without the column of ones, the model would only have the mx_i part.

The parameter vector

Now collect the intercept and slope into one vector:

$$\beta = \begin{bmatrix} c \\ m \end{bmatrix}$$

Different fields use slightly different words for this vector:

- **parameters**
- **coefficients**
- **weights**

For this notebook, we'll mostly say **parameters** or **coefficients**.

The parameter vector applied to a design matrix by matrix multiplication has output of a matrix that is a list of projections?

```
In [6]: beta = np.array([c, m])
```

```
beta
```

```
Out[6]: array([1.3 , 0.75])
```

Predicting with matrix multiplication

Now the whole prediction step is:

$$\hat{y} = X\beta$$

This produces all predictions at once.

```
In [7]: y_hat_matrix = X @ beta
```

```
y_hat_matrix
```

```
Out[7]: array([2.05, 2.8 , 3.55, 4.3 , 5.05])
```

This should match the direct calculation:

$$\hat{y} = mx + c$$

```
In [8]: comparison = np.column_stack([x, y_hat_direct, y_hat_matrix])
```

```
print("columns: x, direct prediction, matrix prediction")
comparison
```

```
columns: x, direct prediction, matrix prediction
```

```
Out[8]: array([[1. , 2.05, 2.05],
               [2. , 2.8 , 2.8 ],
               [3. , 3.55, 3.55],
               [4. , 4.3 , 4.3 ],
               [5. , 5.05, 5.05]])
```

```
In [9]: np.allclose(y_hat_direct, y_hat_matrix)
```

Out[9]: True

The direct calculation and the matrix calculation are the same.

The point of the matrix version is not that it changes the model.

The point is that it gives us a clean general form:

$$\hat{y} = X\beta$$

That form scales naturally to:

- many observations
- many input features
- many model parameters

Visualising the candidate line

The line below is not necessarily the best line.

It is just the line produced by the candidate parameters:

$$\beta = \begin{bmatrix} c \\ m \end{bmatrix}$$

```
In [10]: fig, ax = plt.subplots(figsize=(5.5, 4))

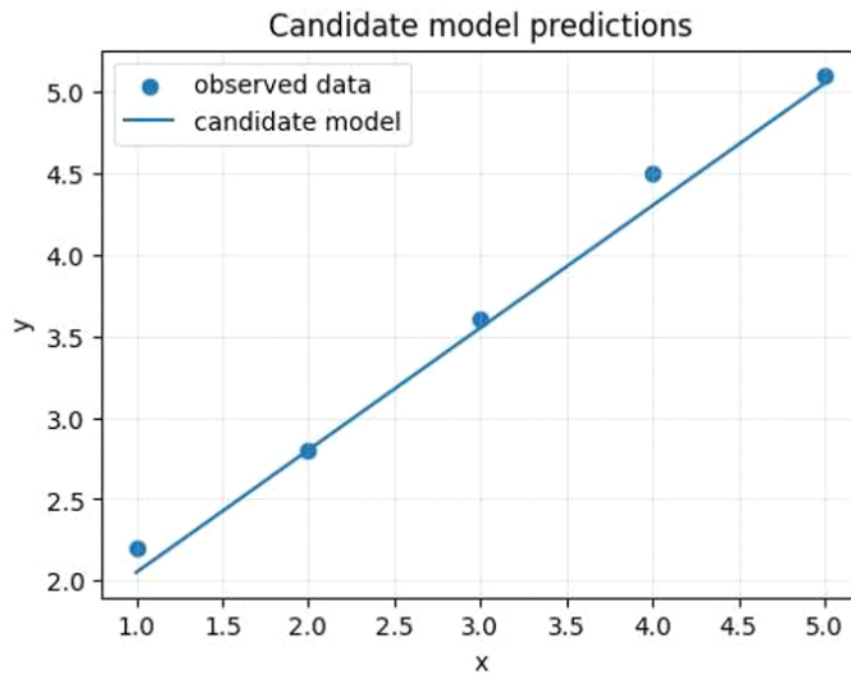
ax.scatter(x, y, label="observed data")

x_line = np.linspace(x.min(), x.max(), 100)
y_line = m * x_line + c

ax.plot(x_line, y_line, label="candidate model")

ax.set_title("Candidate model predictions")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.legend()
ax.grid(True, alpha=0.3)

plt.show()
```



Residuals preview

Once we have predictions, we can compare them to the observed values.

A common definition of residual is:

$$r_i = y_i - \hat{y}_i$$

So:

- positive residual: actual value is above the prediction
- negative residual: actual value is below the prediction
- zero residual: prediction exactly matches the actual value

The full residual vector is:

$$\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}$$

This prepares us for loss functions in the next notebook.

```
In [11]: residuals = y - y_hat_matrix
         residual_table = np.column_stack([x, y, y_hat_matrix, residuals])
         print("columns: x, observed y, predicted y_hat, residual")
         residual_table

columns: x, observed y, predicted y_hat, residual
Out[11]: array([[1. , 2.2 , 2.05, 0.15],
                [2. , 2.8 , 2.8 , 0. ],
                [3. , 3.6 , 3.55, 0.05],
                [4. , 4.5 , 4.3 , 0.2 ],
                [5. , 5.1 , 5.05, 0.05]])
```

```
In [12]: fig, ax = plt.subplots(figsize=(5.5, 4))

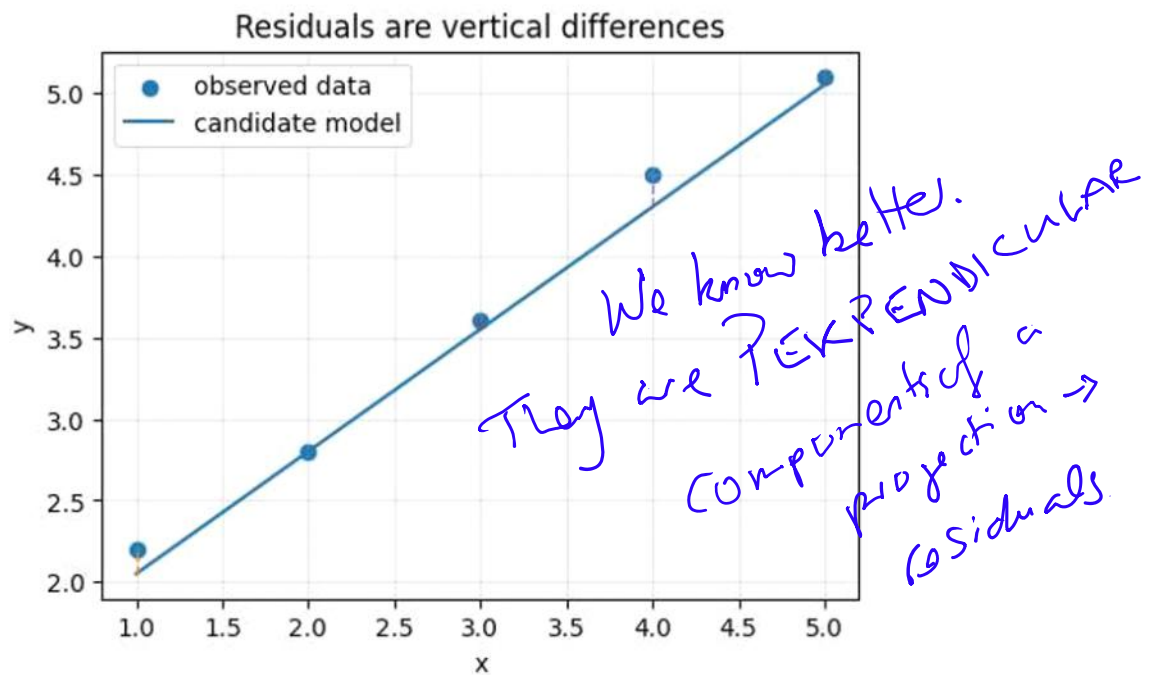
ax.scatter(x, y, label="observed data")

x_line = np.linspace(x.min(), x.max(), 100)
y_line = m * x_line + c
ax.plot(x_line, y_line, label="candidate model")

for x_i, y_i, y_hat_i in zip(x, y, y_hat_matrix):
    ax.plot([x_i, x_i], [y_hat_i, y_i], linestyle="--", linewidth=1)

ax.set_title("Residuals are vertical differences")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.legend()
ax.grid(True, alpha=0.3)

plt.show()
```



Shape check

Before moving on, it is worth checking the shapes.

For:

$$\hat{y} = X\beta$$

we have:

$$X \in \mathbb{R}^{5 \times 2}$$

and:

$$\beta \in \mathbb{R}^2$$

so:

$$\hat{y} \in \mathbb{R}^5$$

In plain English:

- 5 rows in the design matrix
- 2 columns in the design matrix
- 2 parameters
- 5 predictions

```
In [13]: print(f"x shape:      {x.shape}")
         print(f"X shape:      {X.shape}")
         print(f"beta shape:    {beta.shape}")
         print(f"y_hat shape:   {y_hat_matrix.shape}")
```

```
x shape:      (5,)
X shape:      (5, 2)
beta shape:    (2,)
y_hat shape:  (5,)
```

The number of columns in X must match the number of entries in β .

That is the matrix multiplication rule showing up in ML form.

A small manual row check

Let's take the third row of X .

Because Python is zero-indexed, the third row has index `2`.

```
In [14]: row_index = 2

X_row = X[row_index]
x_value = x[row_index]

manual_prediction = X_row[0] * beta[0] + X_row[1] * beta[1]
matrix_prediction = X_row @ beta

print(f"x value: {x_value}")
print(f"design matrix row: {X_row}")
print(f>manual prediction: {manual_prediction}")
print(f"row @ beta prediction: {matrix_prediction}")
```

```
x value: 3.0
design matrix row: [1. 3.]
manual prediction: 3.55
row @ beta prediction: 3.55
```

That one row calculation is the whole idea in miniature:

$$\begin{bmatrix} 1 & x_i \end{bmatrix} \begin{bmatrix} c \\ m \end{bmatrix} = \hat{y}_i$$

The full matrix calculation just performs the same row calculation for every observation.

Preview: more than one feature

Real machine learning datasets usually have more than one input feature.

For example, suppose a prediction depends on two features:

$$\hat{y}_i = b_0 + b_1x_{i1} + b_2x_{i2}$$

Here:

- b_0 is the intercept
- b_1 is the coefficient for feature 1
- b_2 is the coefficient for feature 2
- x_{i1} is feature 1 for observation i
- x_{i2} is feature 2 for observation i

The design matrix becomes:

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} \\ 1 & x_{21} & x_{22} \\ 1 & x_{31} & x_{32} \end{bmatrix}$$

and the parameter vector becomes:

$$\beta = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix}$$

But the prediction equation stays the same:

$$\hat{y} = X\beta$$

```
In [15]: # A tiny two-feature example.
# Each row is one observation.
# Each column after the intercept is one feature.

feature_1 = np.array([1, 2, 3], dtype=float)
feature_2 = np.array([10, 20, 30], dtype=float)

X_two_features = np.column_stack([
    np.ones_like(feature_1),
    feature_1,
    feature_2
])

beta_two_features = np.array([0.5, 2.0, -0.1])

y_hat_two_features = X_two_features @ beta_two_features

print("X:")
print(X_two_features)

print("\nbeta:")
print(beta_two_features)
```

```
print("\npredictions:")
print(y_hat_two_features)
```

```
X:
[[ 1.  1. 10.]
 [ 1.  2. 20.]
 [ 1.  3. 30.]]
```

```
beta:
[ 0.5  2. -0.1]
```

```
predictions:
[1.5 2.5 3.5]
```

Check the first row manually:

$$\hat{y}_1 = b_0 + b_1 x_{11} + b_2 x_{12}$$

Using the numbers above:

$$\hat{y}_1 = 0.5 + 2.0(1) - 0.1(10)$$

So:

$$\hat{y}_1 = 1.5$$

```
In [16]: manual_first_prediction = 0.5 + 2.0 * 1 - 0.1 * 10
manual_first_prediction
```

```
Out[16]: 1.5
```

Again, the point is not the particular numbers.

The point is the structure:

$$\hat{y} = X\beta$$

This is one of the most useful shapes in machine learning.

Tiny exercise

Try these before looking at the answers.

Exercise 1

Suppose:

$$x = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

and the model is:

$$\hat{y} = 3 + 0.5x$$

$$X = \begin{bmatrix} 1 & 2 \\ 1 & 4 \\ 1 & 6 \end{bmatrix}$$

$$\beta = \begin{bmatrix} 3 \\ 0.5 \end{bmatrix}$$

Write the design matrix X and parameter vector β .

Exercise 2

Using your X and β , calculate the three predictions.

Exercise 3

What shape should X have?

2, 3

What shape should β have?

2, 0

What shape should $\hat{y}$ have?

3, 0

$$\hat{y} = X\beta$$
$$= \begin{bmatrix} 1 & 2 \\ 1 & 4 \\ 1 & 6 \end{bmatrix} \begin{bmatrix} 3 \\ 0.5 \end{bmatrix}$$
$$= \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$$

Exercise answers

For Exercise 1:

$$X = \begin{bmatrix} 1 & 2 \\ 1 & 4 \\ 1 & 6 \end{bmatrix}$$

and:

$$\beta = \begin{bmatrix} 3 \\ 0.5 \end{bmatrix}$$

For Exercise 2:

$$\hat{y} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$$

For Exercise 3:

$$X \in \mathbb{R}^{3 \times 2}$$

$$\beta \in \mathbb{R}^2$$

$$\hat{y} \in \mathbb{R}^3$$

Maybe I need this
'language' explained
more

```
In [17]: exercise_x = np.array([2, 4, 6], dtype=float)

exercise_X = np.column_stack([
    np.ones_like(exercise_x),
    exercise_x
])

exercise_beta = np.array([3, 0.5])

exercise_y_hat = exercise_X @ exercise_beta

print("X:")
```

```
print(exercise_X)

print("\nbeta:")
print(exercise_beta)

print("\npredictions:")
print(exercise_y_hat)
```

```
X:
[[1. 2.]
 [1. 4.]
 [1. 6.]]
```

```
beta:
[3.  0.5]
```

```
predictions:
[4. 5. 6.]
```

Summary

The key idea in this notebook is:

$$\hat{y} = X\beta$$

This compact expression means:

arrange the input data as a design matrix, arrange the model coefficients as a parameter vector, and use matrix multiplication to produce predictions.

Important takeaways:

- Each row of X represents one observation.
- Each column of X represents one feature or model term.
- The column of ones allows the model to include an intercept.
- The parameter vector β contains the model coefficients.
- Matrix multiplication gives all predictions at once.
- Residuals compare observed values with predicted values:

$$r = y - \hat{y}$$

The next notebook uses those residuals to ask:

How bad are these predictions, and how can we measure that with a single number?